

Advanced Data Management – Week 2

**Distributed Systems: Problems, Failures and Availability in
Databases**

The Problem Is the Model and the Architecture

- The limitations arise from:
 - The relational model with strong dependence on joins
 - The assumption of centralized control and strong consistency
 - The scale-up architecture as the default

When the model and architecture are transferred to large-scale distributed environments, performance, availability, and complexity issues emerge.

RDBMS Were Not Designed for Cloud-Native Environments

- Traditional RDBMS:
 - Were designed for on-premise infrastructures
 - Assume reliable networks and stable hardware
 - Rely on strong consistency and synchronization
-
- In contrast, cloud-native environments require:
 - Distributed operation from the outset
 - Fault tolerance
 - Elastic scalability
 - Automation

This incompatibility explains why new data models and architectures were developed.

The Problem Is NOT SQL – Conclusion

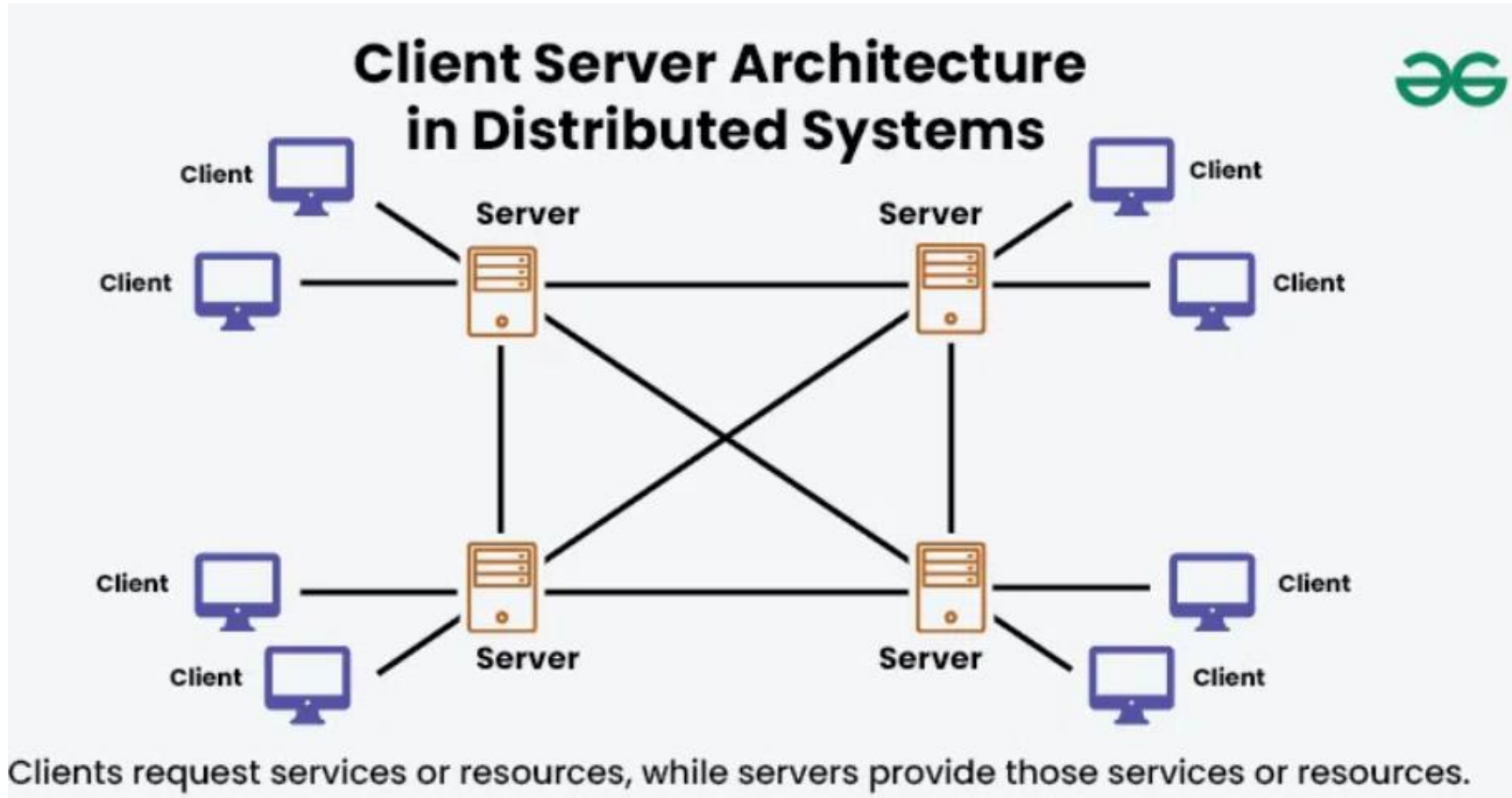
The transition to NoSQL and Big Data is not a rejection of SQL, but a recognition that the traditional relational model and monolithic RDBMS architecture are insufficient for cloud-native, large-scale distributed systems.

Fundamental Principles of Distributed Systems

What Is a Distributed System

- A distributed system is a set of autonomous computational nodes that:
 - Communicate over a network
 - Cooperate to achieve a common goal
 - Appear to the user as a single unified system
-
- Each node has:
 - Its own memory
 - Its own processor
 - Potentially its own storage

A Distributed System



Why Do We Need Distributed Systems

- The use of distributed systems is driven by:
 - The need for horizontal scalability
 - Management of large volumes of data
 - Requirement for high availability
 - Geographic distribution of users and services
- Modern NoSQL systems cannot exist without distributed architecture.

Basic Assumptions

- In a distributed system:
 - The network is not reliable
 - Nodes can fail
 - Latency is not zero
 - There is no global clock

These assumptions are fundamental and determine every design decision.

Distribution Transparency

- The goal of distributed systems is to hide:
 - The physical location of data
 - The number of nodes
 - Failures of individual nodes

The user or application should not know where the data are stored.

Scalability

- Scalability is a fundamental characteristic:
- Vertical (Scale-Up): stronger server
- Horizontal (Scale-Out): more nodes

Distributed NoSQL systems are designed exclusively for scale-out.

Data Partitioning (Sharding)

- Data are divided into segments (shards) and:
 - Stored on different nodes
 - Each shard serves part of the requests
- Goal:
 - Reduce load per node
 - Enable parallel request handling

Data Replication

- Each piece of data:
 - Is stored in multiple replicas
 - Across different nodes
- Benefits:
 - Fault tolerance
 - High availability
 - Faster access
- Cost:
 - Increased complexity in maintaining consistency

Fault Tolerance

- The system must continue operating:
 - Even if one or more nodes fail
 - Without loss of critical data
- Achieved through:
 - Replication
 - Recovery mechanisms
 - Automatic load redistribution

State Consistency

- When multiple replicas exist:
 - When and how are they synchronized?
 - What does the user observe?
- Distributed systems cannot always guarantee immediate consistency. (This leads directly to the CAP Theorem.)

Communication Between Nodes

- Nodes communicate through:
 - Message passing
 - RPC / APIs
 - Asynchronous protocols
- Communication:
 - Has latency
 - May fail
 - Is not always symmetric

Coordination & Agreement

- **In many scenarios, systems require:**
 - Agreement on system state
 - Leader election
 - Synchronization of actions
- This is one of the most difficult problems in distributed systems.

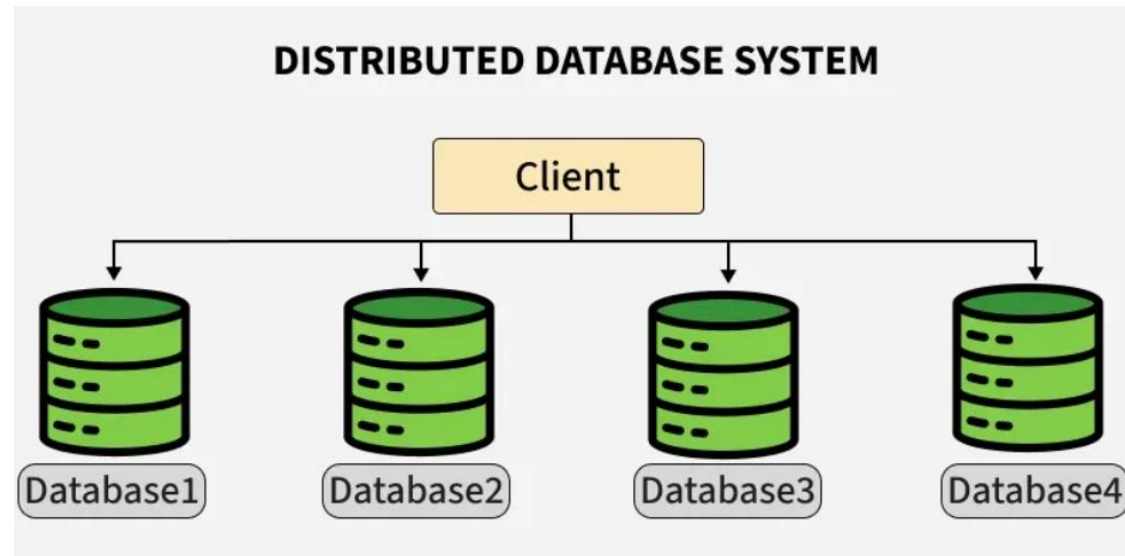
Why All This Matters for NoSQL

- NoSQL databases:
 - Are inherently distributed
 - Rely on partitioning and replication
 - Operate under network uncertainty
-
- Without understanding distributed systems, proper NoSQL design is impossible.

Birth & Philosophy of NoSQL

The Birth of NoSQL

- The emergence of NoSQL systems did not originate from academic theory, but from real, pressing, large-scale demands.
- In the mid-2000s, some of the world's largest technology organizations faced problems that traditional RDBMS could not efficiently solve.



Google, Amazon, Facebook

- Companies such as Google, Amazon, and Facebook managed:
- Billions of users
- Massive data volumes
- Globally distributed infrastructures
- Extremely high read and write loads

These requirements exceeded the limits of classical relational databases.

Need for Massive Scalability

- The core problem was massive scalability:
- Horizontal expansion to hundreds or thousands of nodes
- Continuous addition of resources without downtime
- Tolerance to node and network failures
- Predictable performance at global scale

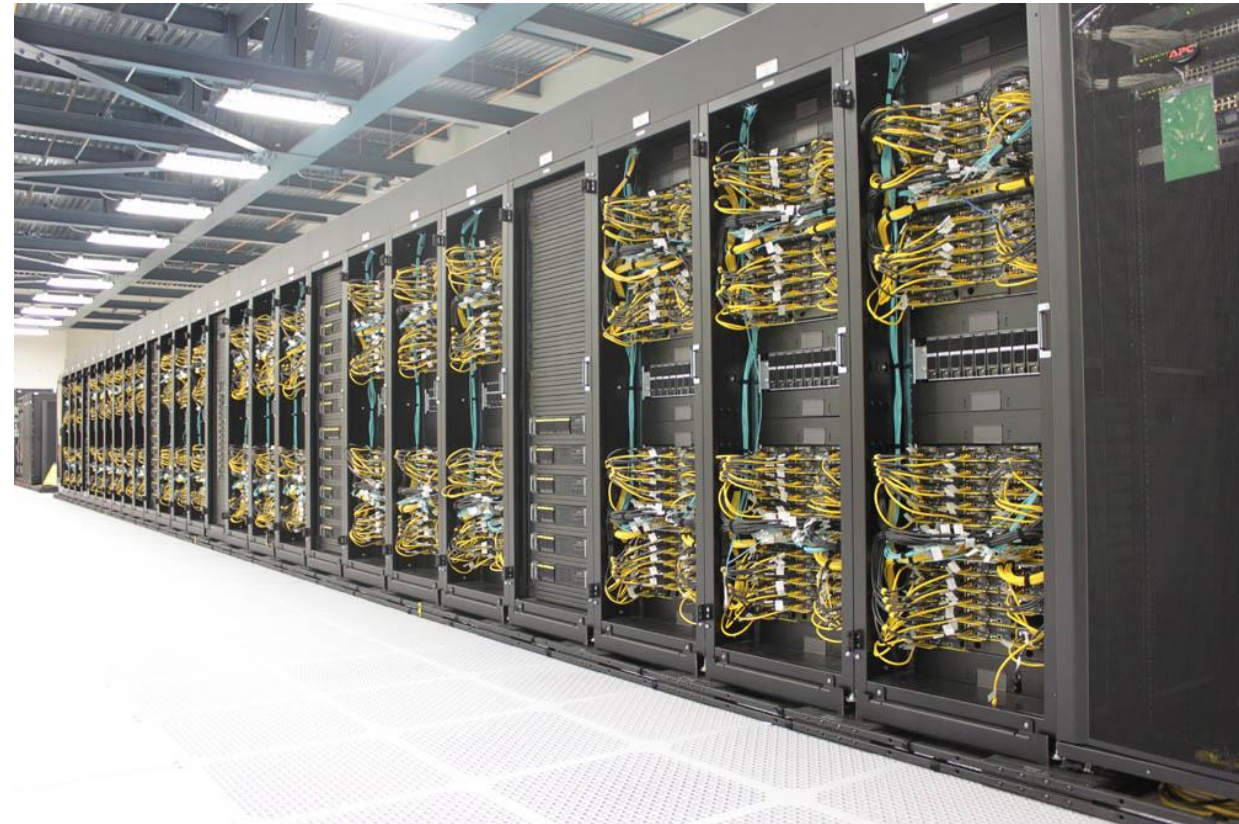
NoSQL systems were designed from the outset for scale-out.

Commodity Hardware

- Another crucial factor was the use of commodity hardware:
 - Cheap, standard servers
 - Accepting failure as normal
 - Shifting complexity from hardware to software
-
- Instead of few powerful servers, NoSQL architectures rely on many simple nodes.

Conclusion

- NoSQL systems were born as a response to global-scale requirements.
- **They were not designed to replace RDBMS entirely**, but to address massive scalability, distributed operation, and cost challenges.



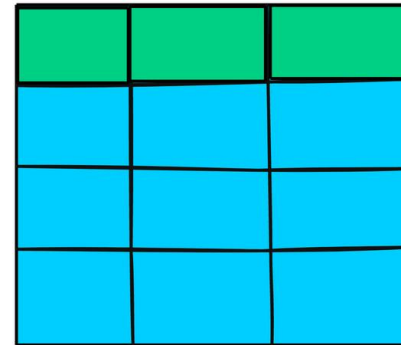
What Does NoSQL Mean

- The term NoSQL is often misunderstood as rejection of SQL.
- In reality, it represents an architectural and philosophical shift driven by **large-scale distributed application** needs.

Not Only SQL

- NoSQL primarily means “Not Only SQL”:
 - Not rejection of SQL
 - Not devaluation of relational databases
 - Expansion of available data models
- Different workloads require different structures and guarantees.

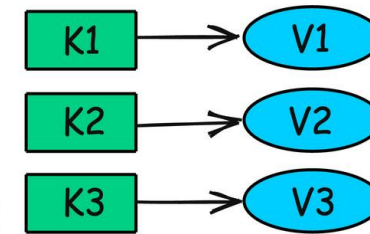
SQL



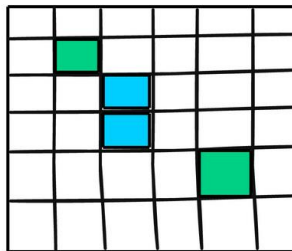
Relational

blog.algomaster.io

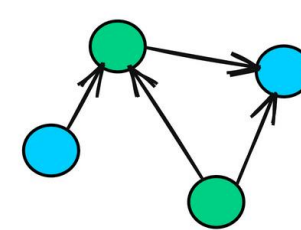
NoSQL



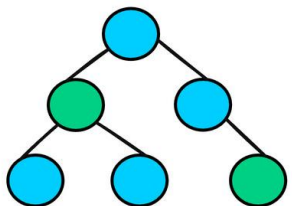
Key-Value



Column Store



Graph



Document

Not Replacement of RDBMS

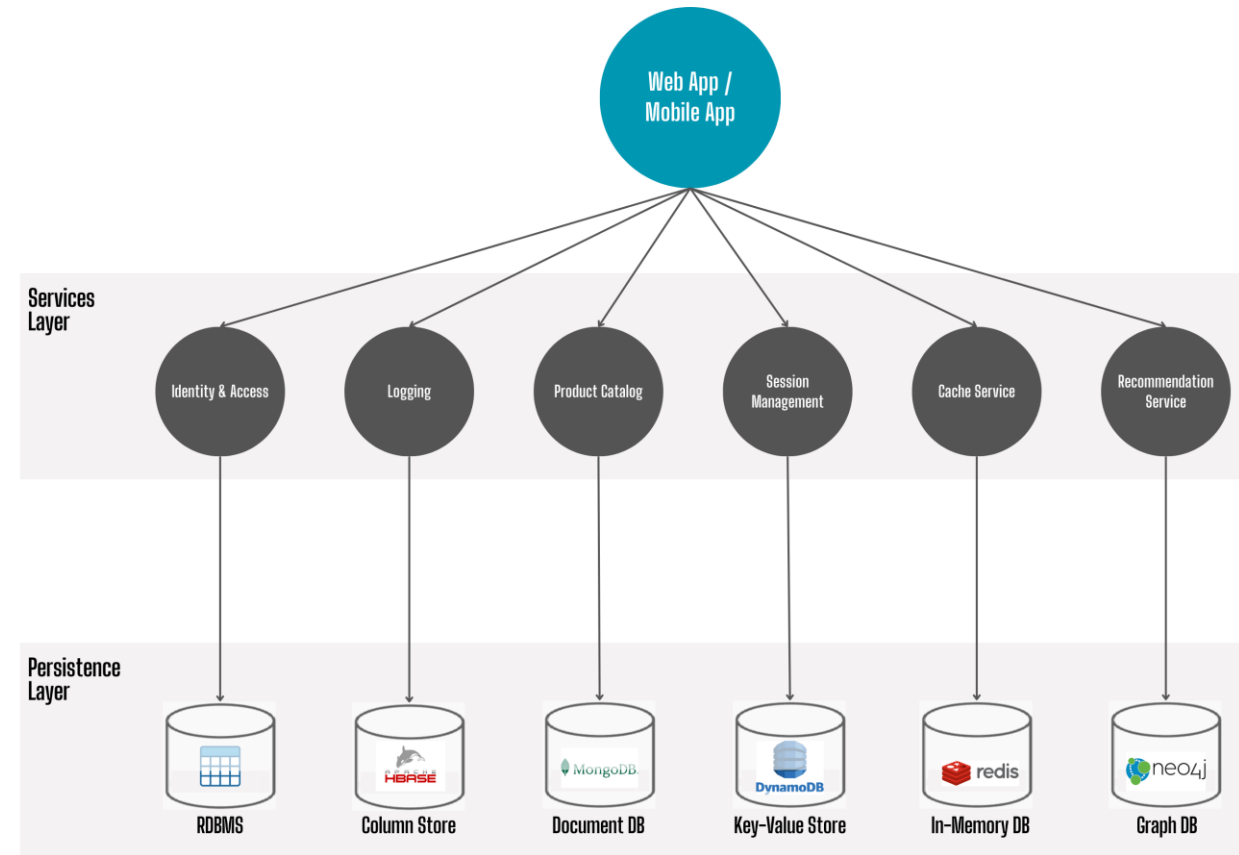
- NoSQL systems:
 - Were not designed to replace RDBMS
 - Do not target all use cases
 - Do not guarantee full ACID in all scenarios
- RDBMS remain irreplaceable in:
 - Transactional systems
 - Financial applications
 - Environments requiring strong consistency

Complementary Technology

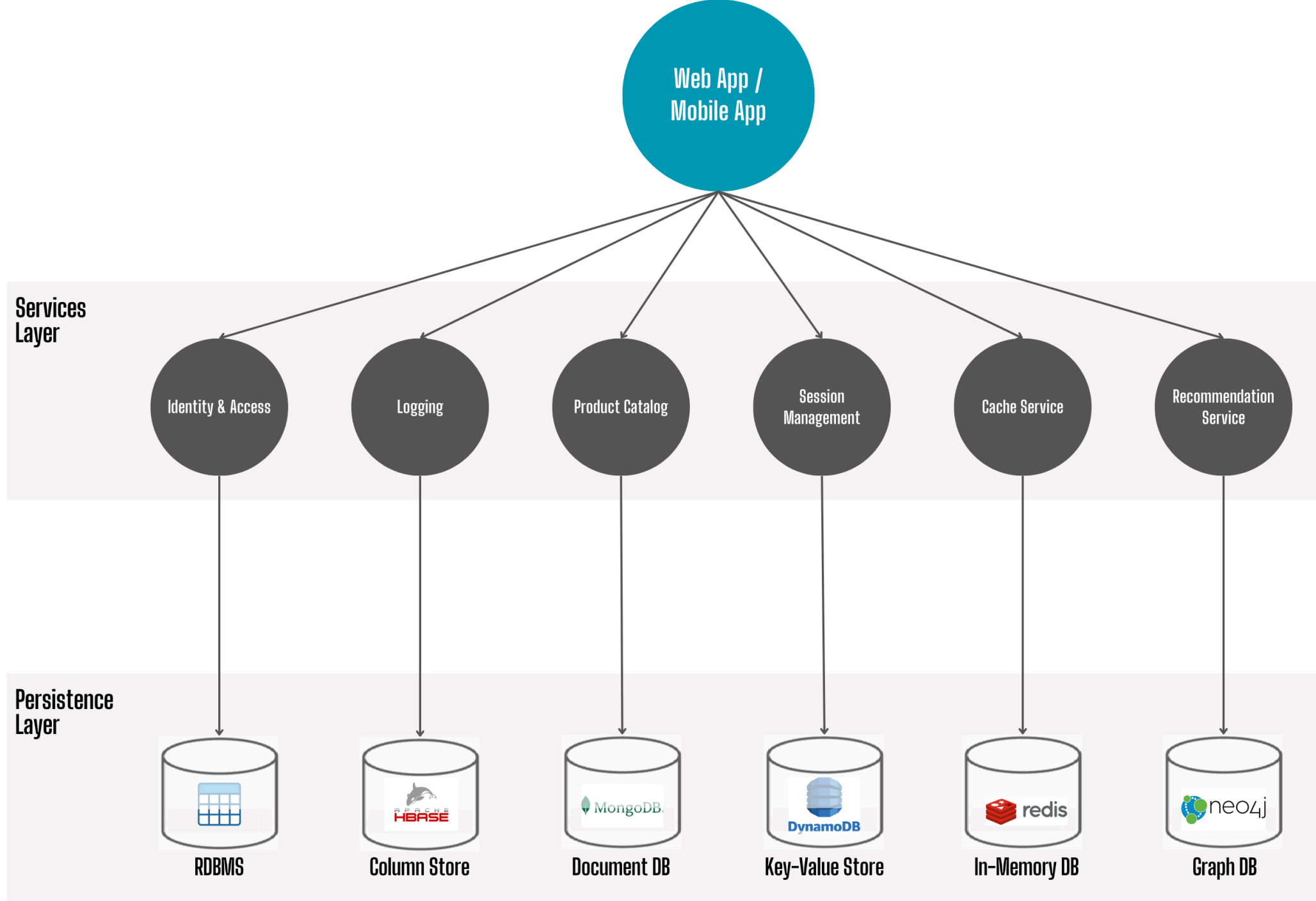
- The role of NoSQL is complementary:
 - Provide massive scalability
 - Support flexible or schema-less data
 - Operate efficiently in distributed cloud-native environments
- Modern architecture follows polyglot persistence.

Conclusions

- NoSQL is not a revolution against RDBMS, but an evolution of the data ecosystem.
- There is no single database model for all problems, but a set of specialized technologies working complementarily.



Polyglot Persistence



Core Principles of NoSQL

- NoSQL systems are based on fundamentally different design principles, oriented from the outset toward distributed, large-scale, cloud-native environments.

Distributed by Design

- NoSQL systems are distributed by nature:
- No central node
- Operate on multiple equal servers
- Assume network and node failures

Distribution is not an afterthought, but a structural characteristic.

Horizontal Scalability

- Scaling through scale-out:
 - Adding nodes without downtime
 - Sharding
 - Linear growth in capacity and throughput
- Performance increases through parallelism.

High Availability

- High availability is a primary objective:
 - Data replication across nodes
 - Requests served by healthy replicas
 - Minimal or zero downtime
-
- **NoSQL systems are designed to ensure the service remains available even under fault conditions.**

Fault Tolerance

- Failure tolerance is built-in:
- Node failure does not imply system failure
- Automatic redistribution
- Self-healing mechanisms

"The system continues to operate, even with reduced performance, until full recovery."

NoSQL - Conclusion

- NoSQL systems were designed for a world where:
- Failures are expected
- Scale is massive
- Availability is critical

The principles of distributed by design, horizontal scalability, high availability and fault tolerance constitute the foundation upon which the entire NoSQL ecosystem is built.

Schema Flexibility

- One of the defining features of NoSQL systems is schema flexibility.
- Unlike traditional RDBMS, where schema is strictly predefined, NoSQL adopts dynamic approaches suited for rapidly changing data environments.

Schema-less

- Schema-less does not mean absence of structure, but:
- No strictly enforced central schema
- Different structures per record
- Data stored without predefined format

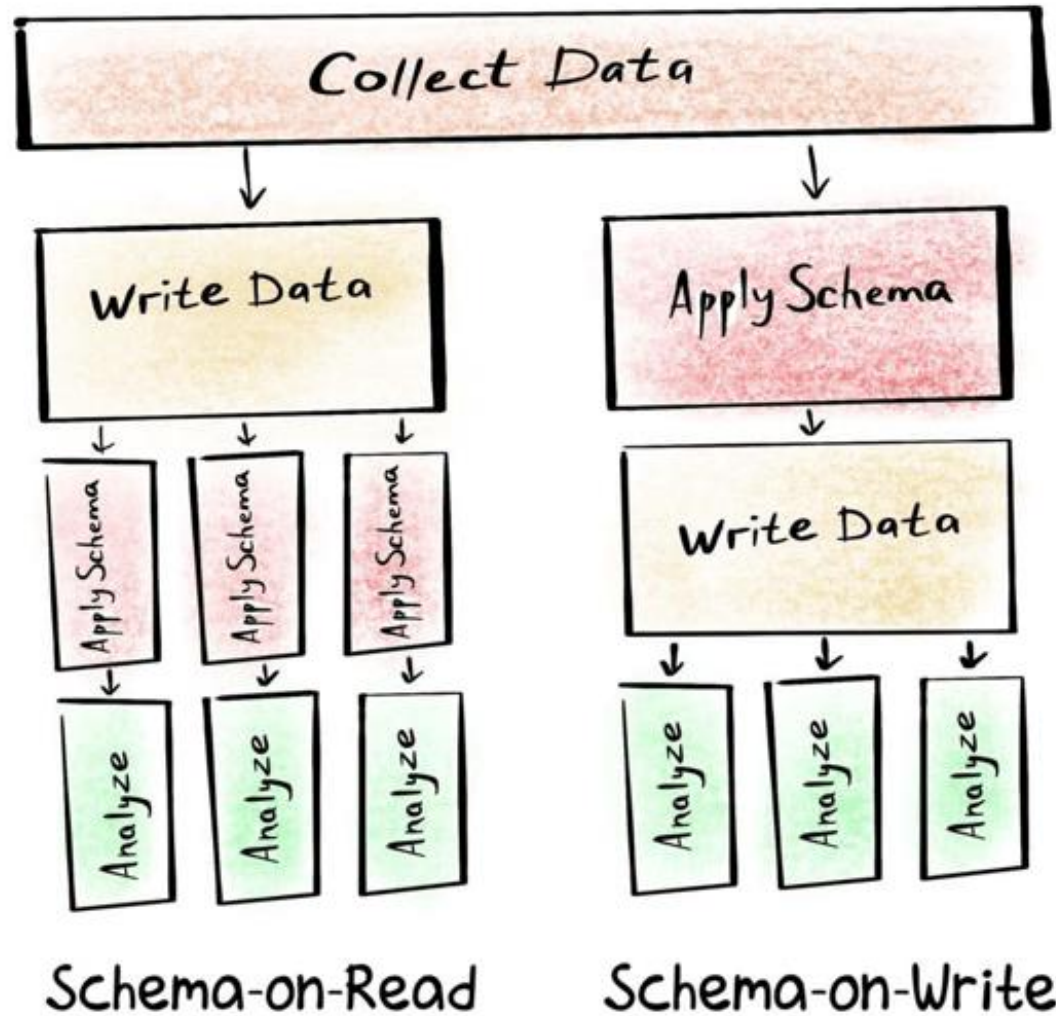
Structure interpretation often moves to the application.

Schema-on-read

- In schema-on-read:
- Data are stored as generated
- Schema applied at read time
- Different applications interpret the same data differently

Especially useful in analytics and Big Data pipelines.

Schema Flexibility

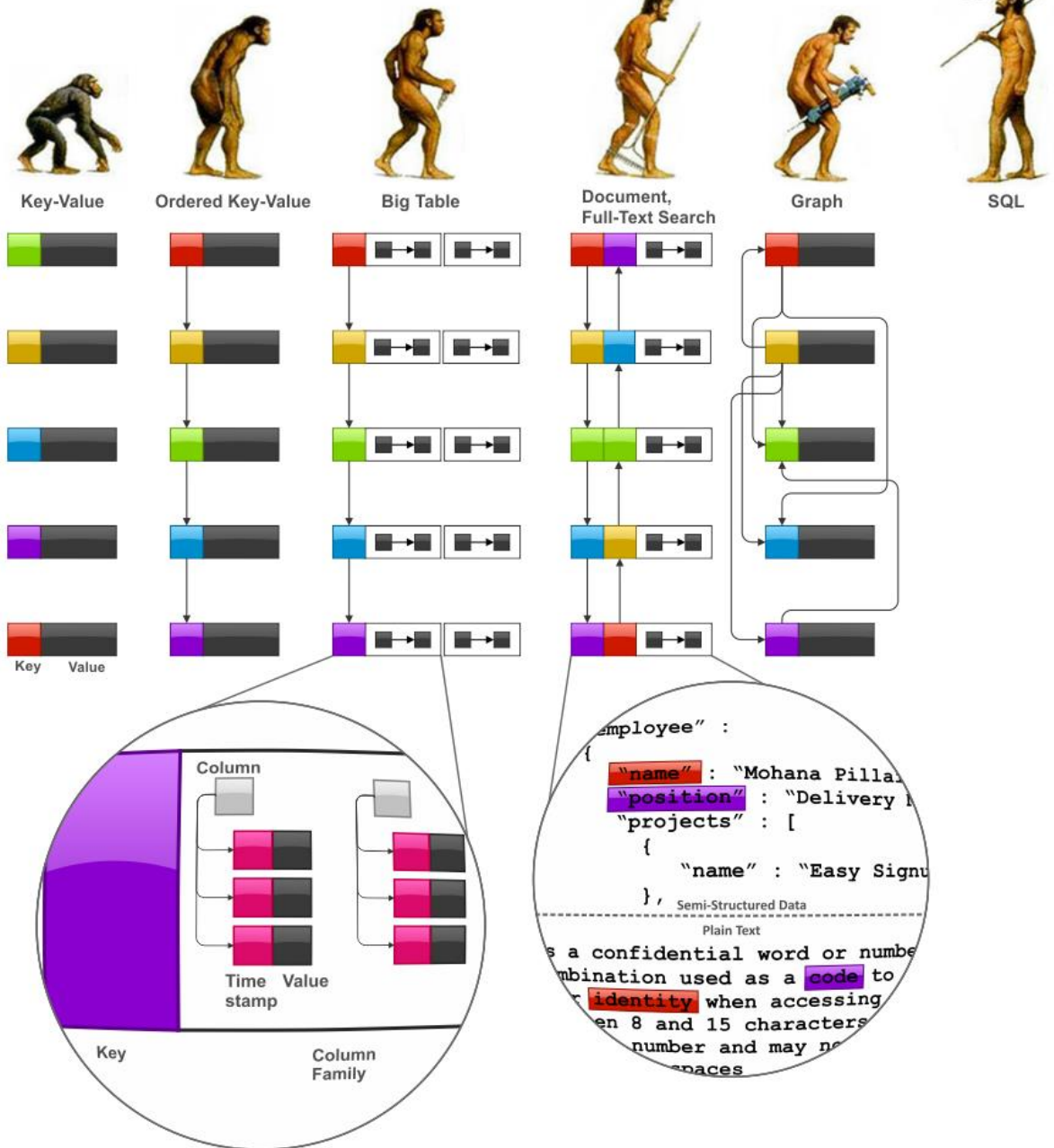


Evolution of Data Models

- Schema flexibility allows:
- Gradual data evolution without downtime
- Easy addition of new fields
- Coexistence of old and new versions

Data models evolve with the application, without heavy migrations or database redesign.

Stop following me, you fucking freaks!



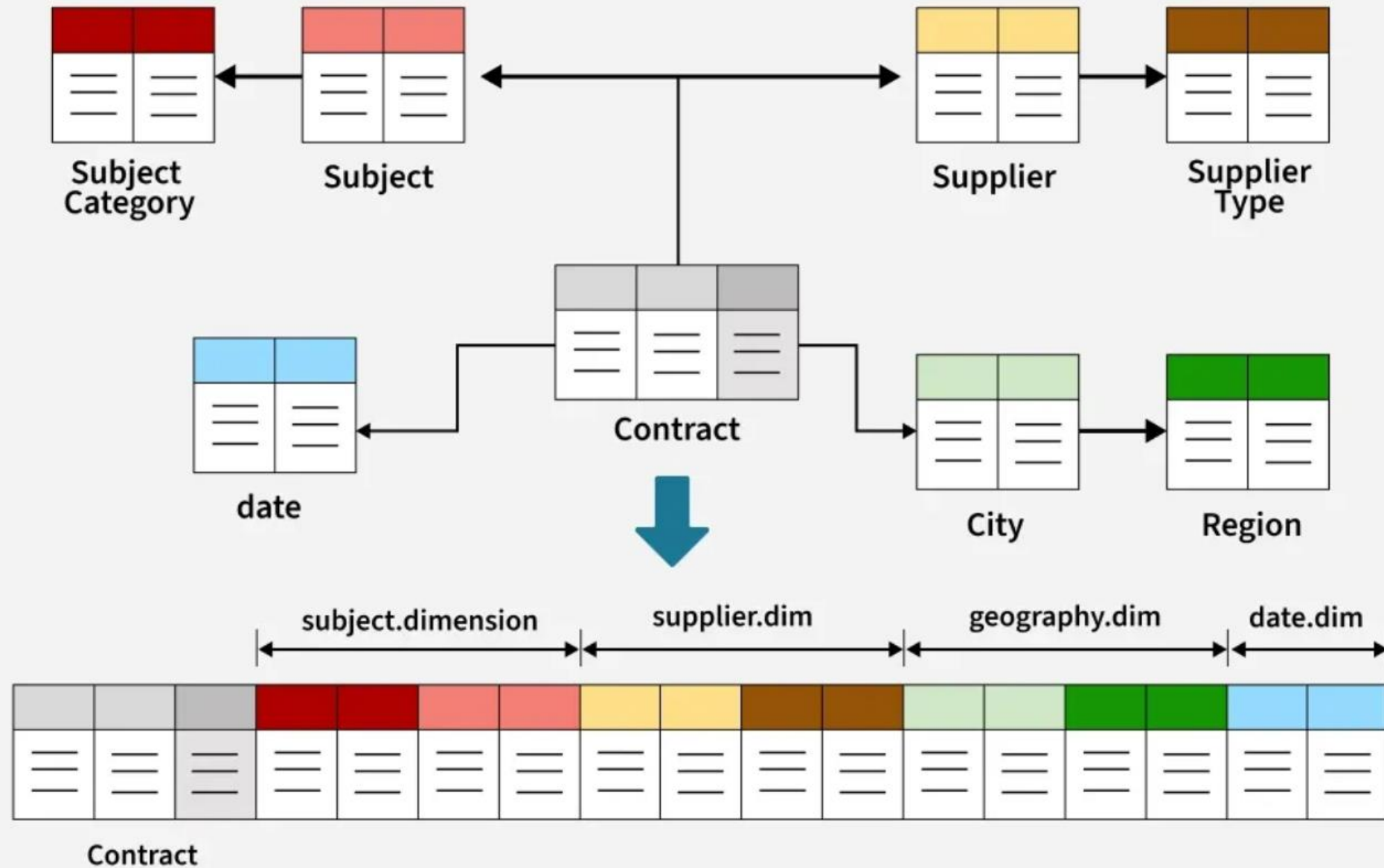
Schema Flexibility - Συμπεράσματα

Schema flexibility in NoSQL systems reflects the reality of modern applications: data changes rapidly and is not always fully known in advance. Schema-less and schema-on-read enable rapid development, experimentation, and adaptability in large-scale, distributed environments.

Denormalization

- Denormalization is a central design choice in NoSQL systems. Unlike the relational model, where normalization and joins are a key mechanism, NoSQL systems adopt a different philosophy, oriented towards performance and scalability.

Denormalization



Why Joins Are Avoided

- In distributed environments:
- Joins require inter-node communication
- Cause heavy data transfer
- Increase latency
- Hinder horizontal scalability

For this reason, NoSQL systems are designed so that a query is served by a single node, without complex read associations.

Denormalization - Replication Δεδομένων

- Avoiding joins leads to replication:
- Related information stored together
- Data repeated intentionally
- Each record contains required read information
- Repetition is a deliberate design choice.

Repetition is a deliberate design choice.

Trade-off: Storage vs Performance

- Denormalization introduces:
- Higher storage usage
- More complex updates
- Much faster reads
- Predictable large-scale performance

In Big Data, storage is cheaper than latency.

Denormalization - Conclusion

Denormalization in NoSQL is not a bad practice, but a necessary strategy for distributed systems. Avoiding joins and controlled data replication allows for high performance, simple scaling, and reliable operation at a global scale.

Consistency vs Availability

In distributed data systems, consistency and availability are two fundamental but often conflicting goals. NoSQL systems introduced different consistency models, accepting that it is not always possible to satisfy all properties simultaneously.

Strong Consistency

- Guarantees:
 - Every read returns the latest write
 - All users see identical state
 - System behaves like a single centralized database
- Trade-off:
 - Higher latency
 - Reduced availability during network failures

Eventual Consistency

- Accepts that:
 - Updates are not immediately visible everywhere
 - Temporary stale reads may occur
 - Nodes eventually converge
-
- Trade-off:
 - Higher availability
 - Lower latency
 - Requires careful application design

Thank You